DESARROLLO WEB EN ENTORNO SERVIDOR · 2.º DAW

Práctica guiada UT3: estructuras de control, arrays, funciones y formularios

Unidad: UT3. Estructuras de control, arrays y formularios	**Modalidad:** Trabajo por parejas (*pair programming*)

1. Dinámica de trabajo

Esta práctica consta de **4 ejercicios secuenciales** que debéis resolver trabajando en pareja bajo la metodología de **conductor / navegante**:

- **Conductor:** es quien escribe el código en el teclado.
- **Navegante:** revisa la sintaxis, consulta los apuntes o el manual oficial de PHP y verifica los resultados en el navegador.

> **Importante:** debéis **intercambiar los roles al finalizar cada ejercicio**, asegurándoos de que ambos integrantes programan exactamente la mitad de la práctica.

2. Estructura y normas de entrega

Cread una carpeta en vuestro servidor local (`htdocs`) que incluya el nombre y apellidos de **ambos integrantes de la pareja** siguiendo el formato `UT3_NombreApellido1_NombreApellido2` (por ejemplo: `UT3_AnaGomez_CarlosRuiz`) con la siguiente estructura exacta de ficheros:

```
UT3_NombreApellido1_NombreApellido2/
├── ejercicio1.php
├── libreria.php
├── ejercicio2.php
├── ejercicio3.php
└── ejercicio4.php
```

- Incluid también un comentario en la cabecera de cada archivo indicando quién ha actuado como **conductor** y quién como **navegante**.
- Todo el código debe estar correctamente **comentado** explicando los bloques principales.
- Al finalizar la sesión, comprimid la carpeta en formato `.zip` manteniendo el mismo nombre con los datos de ambos (`UT3_NombreApellido1_NombreApellido2.zip`) y subidla a la tarea habilitada en Moodle.

3. Enunciados de los ejercicios

Ejercicio 1: condicionales, bucles y operadores de arrays

Roles: integrante A (conductor) | integrante B (navegante) · **Fichero:** `ejercicio1.php`

1. Definid dos arrays asociativos de inventario llamados `$almacen1` y `$almacen2`, donde la clave sea el código del producto (ej. `"P01"`) y el valor las unidades disponibles.
   - *Requisito:* ambos arrays deben compartir al menos un código de producto con distinta cantidad y tener otros códigos diferentes entre sí.
2. Utilizad los operadores `==` (igual) y `===` (idéntico) para comprobar si ambos almacenes son iguales o idénticos, mostrando el resultado con mensajes claros por pantalla.
3. Unid ambos arrays en un tercero llamado `$stockTotal` utilizando el operador `+` y mostrad su estructura completa mediante `print_r()`.
4. Recorred `$stockTotal` mediante un bucle `foreach` extrayendo clave y valor (`$codigo => $unidades`):
   - Si las unidades son `0`, saltad esa iteración utilizando la sentencia `continue`.
   - Utilizad una estructura `if - elseif - else` para mostrar el estado de cada producto:
     - Menos de `5` unidades: *«Stock crítico (reponer urgente)»*.
     - Entre `5` y `15` unidades: *«Stock normal»*.
     - Más de `15` unidades: *«Exceso de stock»*.

Ejercicio 2: librería de funciones, paso por referencia y callbacks

Roles: integrante B (conductor) | integrante A (navegante) · **Ficheros:** `libreria.php` y `ejercicio2.php`

1. En el fichero **`libreria.php`**, programad únicamente las siguientes funciones:
   - **`aplicarDescuento(&$precios, $porcentaje = 10)`**:
     - Recibe un array de precios **por referencia (`&`)** y un porcentaje de descuento con **valor por defecto igual a `10`**.
     - Recorre el array (recordad usar `&` en el `foreach` para modificar los valores reales) y aplica la rebaja correspondiente a cada precio.
   - **`calcularIvaGeneral($importe)`**: devuelve el importe incrementado en un 21 %.
   - **`calcularIvaReducido($importe)`**: devuelve el importe incrementado en un 10 %.
   - **`procesarFactura($funcionIva, $importe)`**: recibe el **nombre de una función como argumento** (*callback*) y un importe, devolviendo el resultado de ejecutar dicha función sobre el importe.
2. En el fichero **`ejercicio2.php`**:
   - Enlazad la librería utilizando `require_once`.
   - Cread un array indexado con 5 precios iniciales (ej. `[100, 250, 50, 80, 120]`).
   - Llamad a `aplicarDescuento()` pasando únicamente el array (para que actúe el valor por defecto) y mostrad el array modificado.
   - Probad `procesarFactura()` llamándola dos veces: una pasándole `"calcularIvaGeneral"` y otra pasándole `"calcularIvaReducido"`, mostrando ambos resultados por pantalla.

Ejercicio 3: control de excepciones (`try / catch / finally`)

Roles: integrante A (conductor) | integrante B (navegante) · **Fichero:** `ejercicio3.php`

1. Programad una función llamada `validarEdadRegistro($edad)` que realice las siguientes comprobaciones:
   - Si el parámetro está vacío (`empty()`) o no es un número (`!is_numeric()`), debe lanzar una excepción con el mensaje: *«Error: el dato introducido no es un valor numérico válido.»*
   - Si `$edad < 18`, debe lanzar otra excepción con el mensaje: *«Error: debes ser mayor de edad para completar el registro.»*
   - Si supera ambas validaciones, la función devolverá `true`.
2. Cread un array de pruebas con distintos escenarios: `$casosPrueba = [25, 15, "abc", "", 40];`.
3. Recorred el array con un bucle `foreach` y, dentro de cada iteración, llamad a `validarEdadRegistro()` dentro de un bloque **`try / catch / finally`**:
   - En el `try`: mostrad el mensaje *«Edad [X] válida: registro permitido.»*
   - En el `catch (Exception $e)`: capturad el error y mostrad el mensaje usando `$e->getMessage()`.
   - En el `finally`: mostrad un separador o texto breve indicando *«Comprobación finalizada»* (debe ejecutarse siempre).

Ejercicio 4: formulario y procesamiento seguro en un solo fichero

Roles: integrante B (conductor) | integrante A (navegante) · **Fichero:** `ejercicio4.php`

1. Construid en la parte inferior del fichero un formulario HTML titulado **«Cálculo de nota media»** que se envíe a sí mismo (`method="POST"` y `action="<?php echo htmlspecialchars($_SERVER["PHP_SELF"]); ?>"`).
   - Campos necesarios: `nombre` (texto), `nota1` (texto/número), `nota2` (texto/número) y un botón de envío.
2. En el bloque PHP superior, comprobad si el formulario ha sido enviado consultando `$_SERVER["REQUEST_METHOD"] == "POST"`.
3. Recuperad los tres campos **sin acceder directamente a `$_POST`**, utilizando obligatoriamente `filter_input(INPUT_POST, ...)`.
4. Validaciones y lógica:
   - Comprobad que ningún campo esté vacío y que ambas notas sean numéricas y estén comprendidas entre `0` y `10`.
   - **Si hay algún error:** mostrad un mensaje de aviso en color rojo y garantizad la **persistencia de los datos**, es decir, los campos del formulario deben conservar lo que el usuario había escrito (`value="..."`).
   - **Si los datos son correctos:** calculad la media aritmética de las dos notas y utilizad una estructura condicional (`if - elseif - else` o `switch`) para mostrar el nombre del alumno, su nota media y su calificación textual:
     - Menor que 5: *suspenso*
     - Entre 5 y 6,99: *aprobado*
     - Entre 7 y 8,99: *notable*
     - Entre 9 y 10: *sobresaliente*